SOFTWARE DESIGN DOCUMENT

Enterprise AI Operations Center (AIOps)

Unified Multi-LLM Gateway, Governance, Observability & Root Cause Analysis Platform

MONISHA- 24BAI1401

TEJAL SELVAM- 24BAI1112

This document converts the project requirements, user stories and wireframe concepts into a structured software design. It explains the design principles, updated high-level architecture, user interface organization, key design decisions, code-level abstractions, maintainability and future evolution.

Core objective: provide a centralized AI control plane where applications use one governed gateway while provider-specific APIs remain hidden behind stable abstractions.

1. Design Principles Applied

The system is designed around abstraction, modularity, high cohesion and low coupling. These principles are supported by Adapter, Factory, Strategy and Chain of Responsibility patterns.

1.1 Abstraction

Provider-specific APIs are hidden behind a common ProviderAdapter interface. The AI Gateway requests an AI response through the interface and does not directly depend on Groq, GPT, Gemini or Claude implementation details.

```
public interface ProviderAdapter {
    ProviderResponse generate(ProviderRequest request);
    ProviderHealth checkHealth();
    String getProviderName();
}
```

1.2 Modularity

The backend is separated into Authentication, AI Gateway, Governance, Routing, Provider Adapter, Cache, Analytics, Audit, Notification and RCA modules. Each module has a defined responsibility and boundary.

1.3 High Cohesion

Closely related responsibilities stay together. Routing contains routing strategies; governance contains policy checks; provider packages contain provider-specific integration. This prevents unrelated logic from being scattered across controllers or services.

1.4 Low Coupling

The gateway depends on interfaces rather than concrete providers. The frontend communicates through REST APIs and does not access database or provider implementation details directly.

1.5 Applied Design Patterns

- Adapter — isolates provider-specific APIs.
- Factory — centralizes provider adapter creation.
- Strategy — makes routing algorithms interchangeable.
- Chain of Responsibility — composes governance checks.

2. High-Level Architecture

The proposed architecture is a modular monolith with layered and component-oriented internal boundaries. It provides a practical deployment model while keeping the software modular enough for future service extraction.

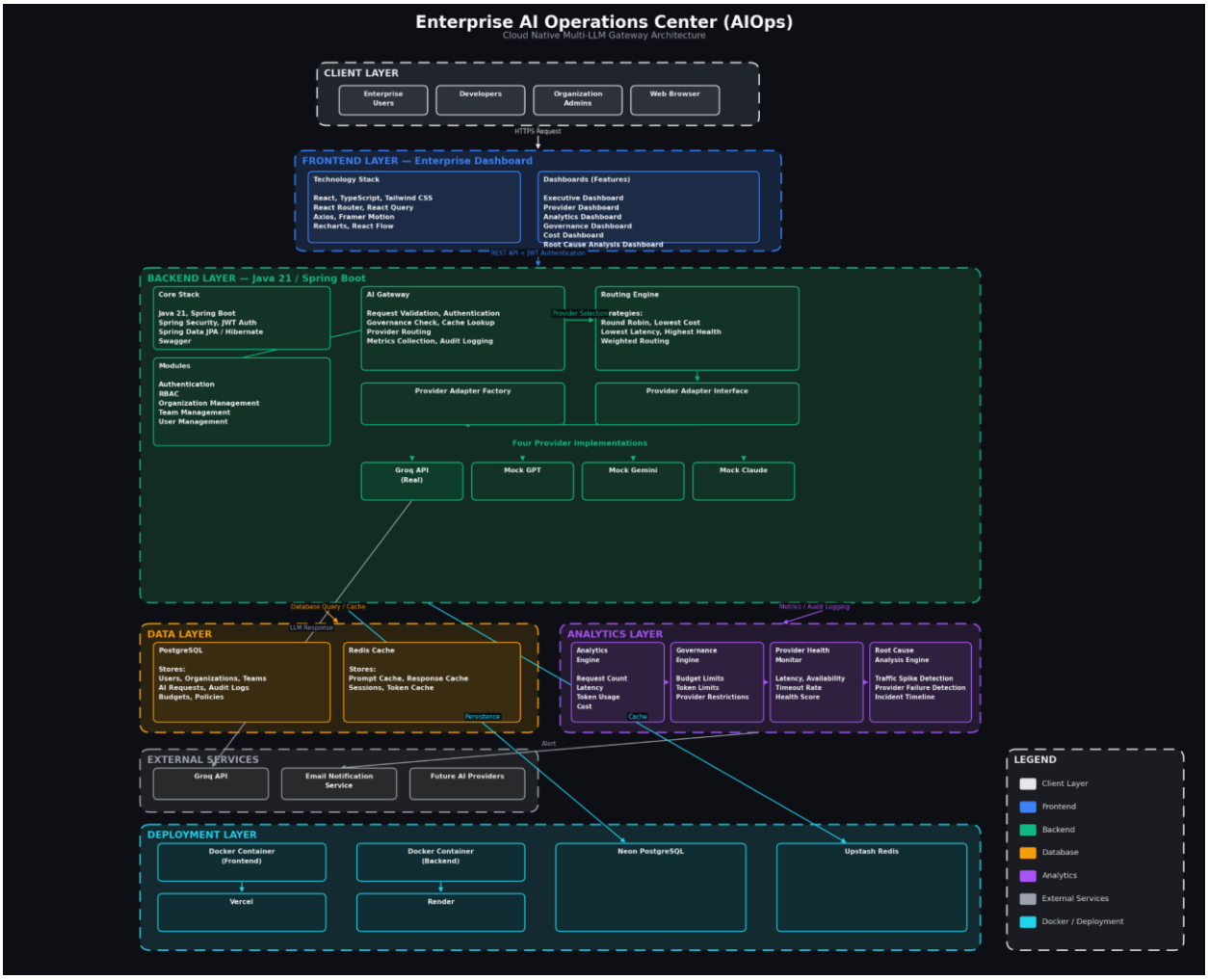


Figure 1. Updated Enterprise AI Operations Center high-level architecture

The updated architecture separates Client, Frontend, Backend, Data, Analytics, External Services and Deployment concerns. The backend contains the AI Gateway, Routing Engine, Provider Adapter Factory and Provider Adapter Interface. The provider layer currently includes one real Groq integration and three mock providers.

3. Architecture Style & Component Interaction

3.1 Architecture style

The architecture follows a modular-monolith style with clear internal layers. React is responsible for presentation, Spring Boot provides the application and domain logic, PostgreSQL provides persistent storage, Redis provides fast cache access, and external provider services are isolated behind adapters.

3.2 Request flow

- 1. A user or enterprise application sends an authenticated request through the frontend/API boundary.
- 2. The AI Gateway validates the request and collects request-level metrics.
- 3. Governance checks access, provider restrictions, token limits and budget policies.
- 4. Redis is checked for a reusable prompt/response result.
- 5. If there is a cache miss, the Routing Engine selects a suitable provider.
- 6. Provider Adapter Factory supplies the appropriate ProviderAdapter implementation.
- 7. The selected adapter communicates with the provider and normalizes the response.
- 8. Response, latency, token and cost information are recorded for analytics and audit.
- 9. Failure and related operational events can later be correlated by the RCA engine.

3.3 Why this architecture?

This structure keeps provider changes away from core business logic, keeps the deployment manageable, and creates explicit seams for testing. If a module later needs independent scaling, the stable boundaries make extraction into a separate service more achievable.

3.4 Data and deployment

PostgreSQL stores users, organizations, teams, AI requests, audit logs, budgets and policies. Redis stores prompt/response and session-related cache data. The deployment design uses Docker containers with Vercel for the frontend, Render for the backend, Neon PostgreSQL and Upstash Redis.

4. Backend Code Structure & Patterns

4.1 Package organization

```
aiops/
├── auth/
├── gateway/
├── governance/
├── routing/
│   ├── RoutingStrategy.java
│   ├── RoundRobinStrategy.java
│   ├── LowestCostStrategy.java
│   └── HighestHealthStrategy.java
├── provider/
│   ├── ProviderAdapter.java
│   ├── GroqProviderAdapter.java
│   ├── MockGptAdapter.java
│   ├── MockGeminiAdapter.java
│   ├── MockClaudeAdapter.java
│   └── ProviderFactory.java
├── cache/
├── analytics/
├── audit/
├── rca/
└── notification/
```

4.2 Strategy Pattern for routing

Routing is variable behavior, so each algorithm is isolated behind RoutingStrategy. The gateway can use Round Robin, Lowest Cost, Lowest Latency, Highest Health or Weighted Routing without rewriting its orchestration logic.

```
public interface RoutingStrategy {
    ProviderAdapter select(List<ProviderAdapter> providers,
                           RoutingContext context);
}
```

4.3 Adapter + Factory

```
public class ProviderFactory {
    public ProviderAdapter getAdapter(String provider) {
        return switch (provider) {
            case "GROQ" -> new GroqProviderAdapter();
            case "MOCK_GPT" -> new MockGptAdapter();
            case "MOCK_GEMINI" -> new MockGeminiAdapter();
            case "MOCK_CLAUDE" -> new MockClaudeAdapter();
            default -> throw new IllegalArgumentException("Unknown provider");
        };
    }
}
```

5. Governance, Cache, Observability & RCA

5.1 Governance

Governance is positioned before provider execution. A request can be evaluated against access policies, provider restrictions, token limits, budgets and other configured rules.

```
public interface GovernanceRule {
    GovernanceResult evaluate(AiRequest request,
                              GovernanceContext context);
}
```

Multiple rules can be composed in a Chain of Responsibility-style flow. This makes policy behavior easier to extend than a single large conditional service.

5.2 Redis cache

A normalized key can be generated from model, prompt and relevant parameters. A cache hit returns the stored response without another provider call. A miss continues through routing and provider execution and may be stored with a TTL.

```
String cacheKey = hash(request.model(),
                           request.prompt(),
                           request.parameters());

Optional<CachedResponse> cached = cache.get(cacheKey);
if (cached.isPresent()) {
    metrics.recordCacheHit();
    return cached.get().response();
}
```

5.3 Observability

Operational records should include request ID, timestamp, selected provider, latency, token usage, estimated cost and success/failure status. Provider health uses latency, failures, availability and timeout behavior.

5.4 Root Cause Analysis

RCA correlates events into an incident chain such as Traffic Spike → Provider Overload → Latency Increase → Timeouts → Fallback. The RCA screen should present the timeline, impact, affected requests and recommended action.

6. User Interface Design

The six primary Figma screens are organized around operational tasks. A shared navigation shell and consistent visual language allow users to move from high-level status to detailed investigation.

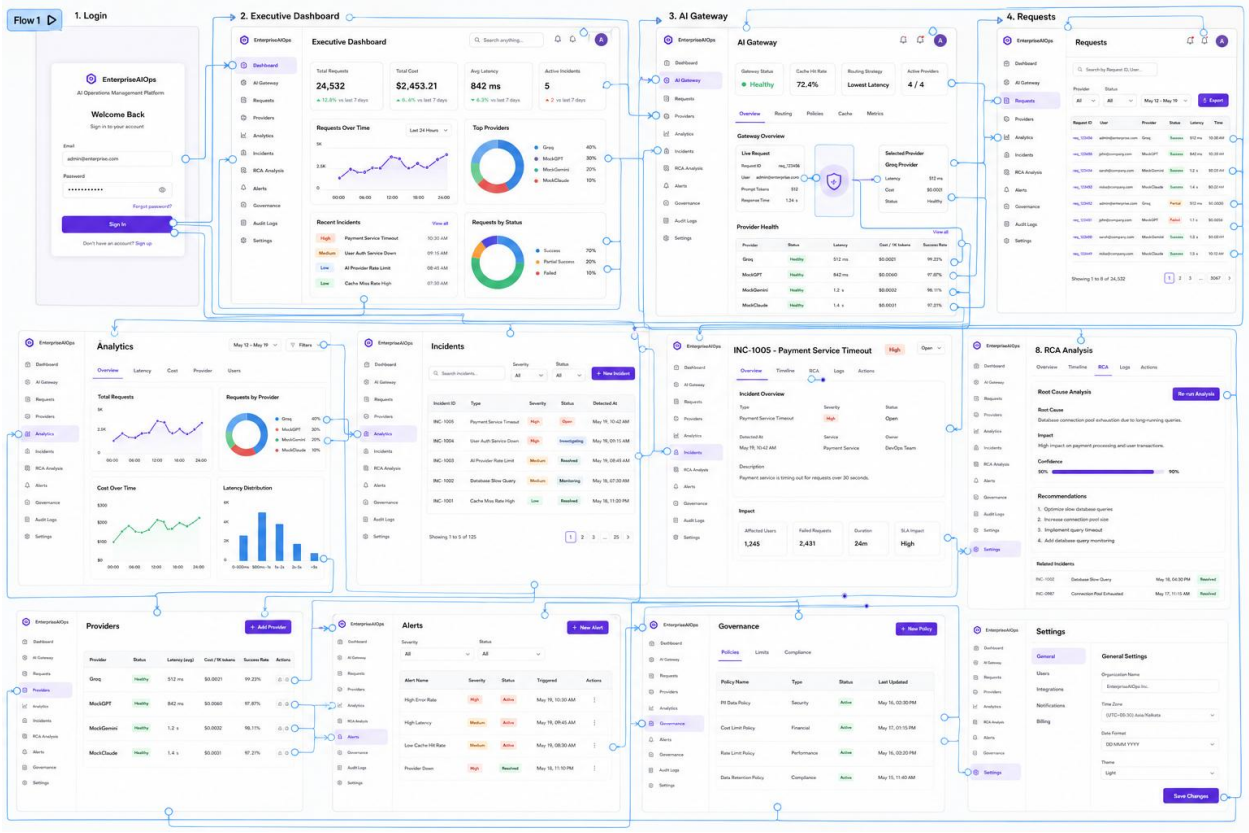


Figure 2. Updated Figma flow and dashboard screens

The prototype shows Login, Executive Dashboard, AI Gateway, Requests, Analytics, Incidents, Incident Detail, RCA Analysis, Providers, Alerts, Governance and Settings views. The flow demonstrates how the main operational screens connect through navigation and actions.

7. UI Usability & Frontend Structure

7.1 User-friendliness

- Clear hierarchy — important health and incident information appears before detailed data.
- Consistency — shared navigation, cards, tables, filters and action placement reduce cognitive load.
- Progressive disclosure — summaries are shown first and detailed request/incident information is available when needed.
- Actionable information — abnormal metrics connect to provider, incident or RCA investigation paths.
- Readable visualization — charts use labels, legends and meaningful units.
- Feedback — loading, success, failure and validation states should be visible.
- Responsive layout — spacing and hierarchy should remain usable across common screen sizes.

7.2 Frontend structure

```
frontend/src/
├── components/
│   ├── cards/
│   ├── charts/
│   ├── tables/
│   └── layout/
├── pages/
│   ├── Login/
│   ├── ExecutiveDashboard/
│   ├── Gateway/
│   ├── Providers/
│   ├── Analytics/
│   └── RCA/
├── services/api.ts
├── hooks/
├── types/
└── routes/
```

7.3 API boundary

```
export async function submitAiRequest(request: AiRequest) {
  const response = await api.post("/api/gateway/request", request);
  return response.data;
}
```

Reusable UI components prevent duplicated layout logic. The frontend uses API services as its boundary and does not contain provider SDK logic or database concerns.

7.4 Screen responsibilities

- Login — secure entry point.
- Executive Dashboard — overall health and KPIs.
- AI Gateway — routing, cache and request behavior.

- Requests — request-level operational records.
- Analytics — trends and comparative metrics.
- RCA — connected incident evidence and recommendations.

8. Key Design Decisions & Why

Decision 1 — Modular Monolith

Provides strong internal separation while keeping development, testing and deployment manageable. Clear boundaries preserve a path to later service extraction.

Decision 2 — Provider Adapter Interface

AI providers expose different APIs. A common adapter prevents vendor-specific details from spreading through the gateway and makes new providers easier to add.

Decision 3 — Strategy Pattern for Routing

Routing criteria can change. Encapsulating algorithms lets the platform add new policies without rewriting gateway orchestration.

Decision 4 — Redis Prompt/Response Cache

Fast cache access reduces repeated provider calls and separates short-lived cache data from persistent relational data.

Decision 5 — Event-Centered RCA

Operational failures are easier to understand as related events. Correlation supports timelines, impact analysis and recommended actions.

8.1 Maintainability example

If a new provider is introduced, the main change is to implement ProviderAdapter, register the implementation with the factory/configuration and provide its credentials. The gateway, governance and UI contracts can remain stable.

8.2 Routing evolution

If the project later requires Quality-Adjusted Cost Routing, a new RoutingStrategy implementation can be introduced while existing strategies remain unchanged.

8.3 UI evolution

A new dashboard can reuse the existing application shell, metric cards, chart components and API service conventions. This reduces duplication and keeps the interface consistent.

9. Maintainability, Testing & Future Evolution

9.1 Testing approach

- Unit tests — routing strategies, governance rules, cache-key generation and provider adapters.
- Integration tests — gateway-to-provider flow, Redis behavior and PostgreSQL persistence.
- Security tests — authentication, authorization and restricted operations.
- UI tests — login, dashboard loading, gateway requests and RCA navigation.
- Failure tests — provider timeout/failure, cache miss and fallback scenarios.

9.2 Future evolution

- Add additional real AI providers through new adapter implementations.
- Introduce asynchronous event processing for higher-volume observability.
- Extract selected modules into independent services when scale requires it.
- Add richer policy management and organization-level budgets.
- Expand RCA with stronger event correlation and historical incident comparison.
- Introduce additional dashboards and configurable alert rules.

9.3 Conclusion

The proposed software design turns the Enterprise AI Operations Center into a structured, maintainable system. Abstraction isolates external providers, modularity separates responsibilities, high cohesion keeps modules focused, and low coupling limits the impact of change. The architecture and UI are structured so future providers, routing strategies, governance rules, analytics features and operational workflows can be added without unnecessary rewrites.

10. GitHub Repo Link

https://github.com/moni182006-collab/enterprise-ai-operations-center.git